

Gradient Descent for Regularized Linear Regression

This section explains how to adapt the gradient descent algorithm to work with the new, regularized cost function for linear regression.

The Regularized Cost Function (Recap)

The cost function for regularized linear regression includes an additional term to penalize large parameter values:

$$J(\vec{w}, b) = \frac{1}{2m} \sum_{i=1}^m (f_{\vec{w},b}(\vec{x}^{(i)}) - y^{(i)})^2 + \frac{\lambda}{2m} \sum_{j=1}^n w_j^2$$

Updated Gradient Descent Algorithm

To minimize this new cost function, we need to update the partial derivative terms used in the gradient descent update rules.

- **Derivative with respect to w_j :**
 - The original derivative is modified by an additional regularization term.
 - $\frac{\partial J}{\partial w_j} = \left(\frac{1}{m} \sum_{i=1}^m (f_{\vec{w},b}(\vec{x}^{(i)}) - y^{(i)}) x_j^{(i)} \right) + \frac{\lambda}{m} w_j$
 - **Derivative with respect to b :**
 - Since the bias term b is not regularized, its derivative remains **unchanged**.
 - $\frac{\partial J}{\partial b} = \frac{1}{m} \sum_{i=1}^m (f_{\vec{w},b}(\vec{x}^{(i)}) - y^{(i)})$
-

Implementation of the Update Rules 🧑🏻💻

The complete gradient descent algorithm for regularized linear regression involves repeatedly performing the following **simultaneous updates**:

- **Update for w_j :**
$$w_j := w_j - \alpha \left[\left(\frac{1}{m} \sum_{i=1}^m (f_{\vec{w},b}(\vec{x}^{(i)}) - y^{(i)}) x_j^{(i)} \right) + \frac{\lambda}{m} w_j \right]$$
 - **Update for b :**
$$b := b - \alpha \left[\frac{1}{m} \sum_{i=1}^m (f_{\vec{w},b}(\vec{x}^{(i)}) - y^{(i)}) \right]$$
-

Intuition Behind the Update (Optional)

We can rewrite the update rule for w_j to better understand what regularization is doing on each step:

$$w_j := w_j \left(1 - \alpha \frac{\lambda}{m} \right) - \alpha \frac{1}{m} \sum_{i=1}^m (f_{\vec{w},b}(\vec{x}^{(i)}) - y^{(i)}) x_j^{(i)}$$

- The term $(1 - \alpha \frac{\lambda}{m})$ is a number that is typically just **slightly less than 1** (e.g., 0.999).
- This means that on every iteration, before subtracting the usual gradient term, the algorithm first **multiplies w_j by a number less than one**, which has the effect of **shrinking the weight** slightly.
- This constant shrinking effect is how regularization prevents the parameters from growing too large.